

ENTERPRISE OPERATIONS INTELLIGENCE PLATFORM

Enterprise Operations Command Center • EOCC

AI-Powered Enterprise Operations Intelligence Platform

From Monitoring Systems to Operational Intelligence

Most enterprises know something failed.

Very few know why it failed, who owns it, what business processes are impacted, and what should happen next.

**EOCC
Intelligence Layer**

sits above



Existing Enterprise Tools

**AWS • Splunk • Jira
Confluence • GitHub**

THE REALITY

The Enterprise Operations Reality

Every enterprise has the tools. The answers are trapped between them.

AWS

Infra metrics
& alarms

Splunk

Logs &
signals

Jira

Tickets &
ownership

Confluence

Runbooks
& docs

GitHub

Code &
changes

MONITORING EXISTS. OPERATIONAL INTELLIGENCE DOES NOT.

?

What
happened?

?

Why did it
happen?

?

Who
owns it?

?

What's
impacted?

?

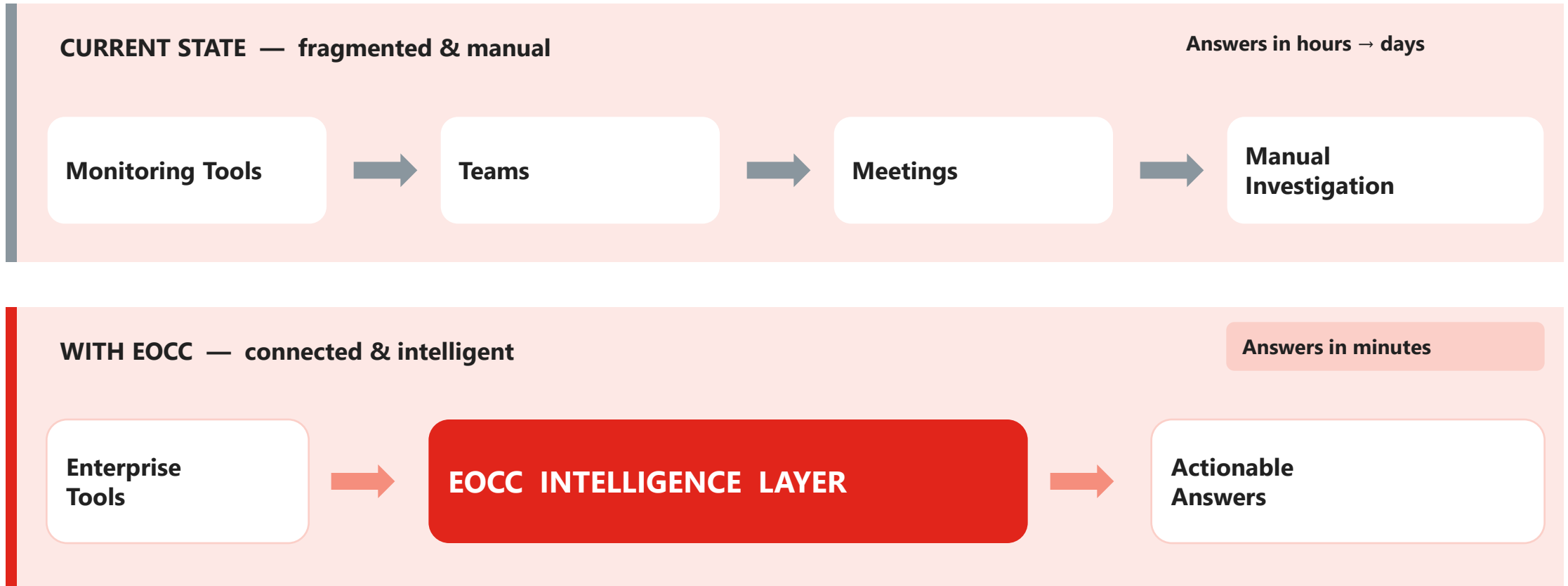
What do we
do next?

Each answer lives in a different system. Today a human stitches them together — manually, under pressure, mid-incident.

THE THESIS

The Missing Layer

Monitoring tells you something broke. Intelligence tells you what to do about it.



EOCC sits above your existing tools. It does not replace them — it connects them.

THE DEFINITION

What EOCC Actually Is

Not a tool. A layer of intelligence over the tools you already run.

EOCC · Enterprise Operations Intelligence Platform

Operational Intelligence

Health ranked by business risk

Ownership Intelligence

Who owns it, who escalates

Dependency Intelligence

Blast radius across services

Business Impact Intelligence

Infra mapped to capabilities

Investigation Intelligence

Automated root-cause correlation

Decision Support Intelligence

Cited, recommended next steps

▲ reads from your existing systems (read-only)

AWS

Splunk

Jira

Confluence

GitHub

EOCC connects them into one intelligence layer — it does not replace them.

THE OPERATING MODEL

The Five Questions Every Operations Team Needs Answered

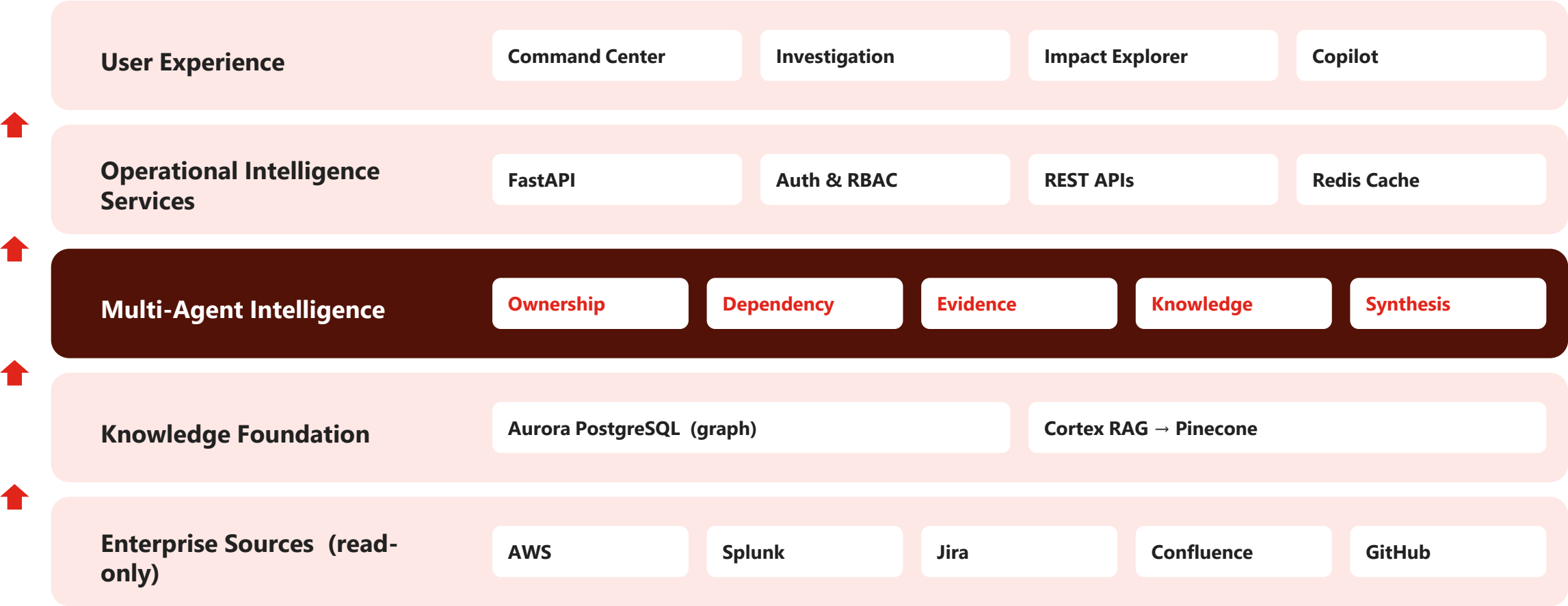
One question. One capability. One source of truth.

OPERATIONS ASKS		EOCC ANSWERS WITH	FROM
1	What is happening?	 Operational Intelligence Health ranked by business risk	AWS · Splunk
2	Why is it happening?	 Investigation Intelligence Evidence + knowledge correlation	Splunk · Jira · Docs
3	What is the business impact?	 Business Impact Intelligence Blast radius → business capability	Aurora graph
4	Who owns it?	 Ownership Intelligence Owner, team & escalation chain	Aurora
5	What should we do next?	 Decision Support Intelligence Cited, recommended next action	Cortex AI

THE ARCHITECTURE

EOCC Architecture Overview

Five layers. Data flows up; intelligence comes back down.



Diagnostic logic lives in the agents — not in dashboards or human meetings.

THE CORE INNOVATION

The Core Innovation

Most platforms monitor. EOCC reasons — over two complementary brains.

STRUCTURED BRAIN

Aurora Knowledge Graph

Deterministic facts — ownership, dependencies, teams, business mapping. Answered by SQL.

No guessing. No hallucination.



UNSTRUCTURED BRAIN

Cortex AI RAG → Pinecone

Semantic memory — runbooks, RCAs, prior incidents, documentation. Retrieved with citations.

Every claim traced to its source.

MULTI-AGENT AI FRAMEWORK — parallel agents gather · one synthesis step reasons

=

ENTERPRISE OPERATIONAL INTELLIGENCE

Two brains plus reasoning agents produce answers — not another stream of alerts.

THE INTELLIGENCE ENGINE

Multi-Agent Intelligence Framework

Specialist agents reason in parallel; one engine synthesizes the conclusion.

LangGraph Orchestrator — routes the trigger, runs agents in parallel (15s budget), then joins

Ownership Agent

Owner, team & escalation chain

Deterministic

Dependency Agent

Blast-radius dependency graph

Deterministic

Investigation Agent

Signals, timeline, similar tickets

Evidence

Knowledge Agent

Runbooks & prior RCAs

Cited

Business Impact Agent

Affected capability & severity

Reasoning

AI SYNTHESIS ENGINE — reasons across all agent outputs in one step



InvestigationPackage

Severity · Root cause · Owner · Business impact · Recommended action · Citations

Reasoning, not retrieval — agents draw conclusions; the engine decides what matters.

WORKED EXAMPLE

Example Incident Investigation

Scenario: a critical database latency spike — from alarm to recommended action.

- 1 Detection**
Aurora latency alarm → EventBridge → SQS
- 2 Ownership Discovery**
Owning team + on-call identified
- 3 Dependency Analysis**
Blast radius mapped to depth 3
- 4 Business Impact**
Affected capability + severity ranked
- 5 Historical Similarity**
Matched a prior incident and its fix
- 6 Recommended Action**
Cited next step delivered to engineer

Automatic

Aurora · <500ms

Aurora · <2s

Graph

Cortex RAG

Synthesis

INVESTIGATION TIME

~30 min

manual, across 4 tools

under 30 sec

automated, one package

**4 agents run in parallel,
not 4 humans in a bridge
call.**

THE PILOT

Phase 1 Pilot Scope

Five capabilities. Fully read-only. Deployable as a low-risk pilot.

Operational Health Monitoring

Command Center

Health ranked by business risk

Automated Investigation

Investigation

Root cause in one package

Impact Analysis

Impact Explorer

Blast radius in business terms

Ownership Discovery

Ownership Graph

Owner & escalation, instantly

AI Operations Copilot

Copilot

Plain-language, cited answers

Built for a safe pilot

✓ Read-only

✓ No production changes

✓ Low risk

✓ Pilot-friendly

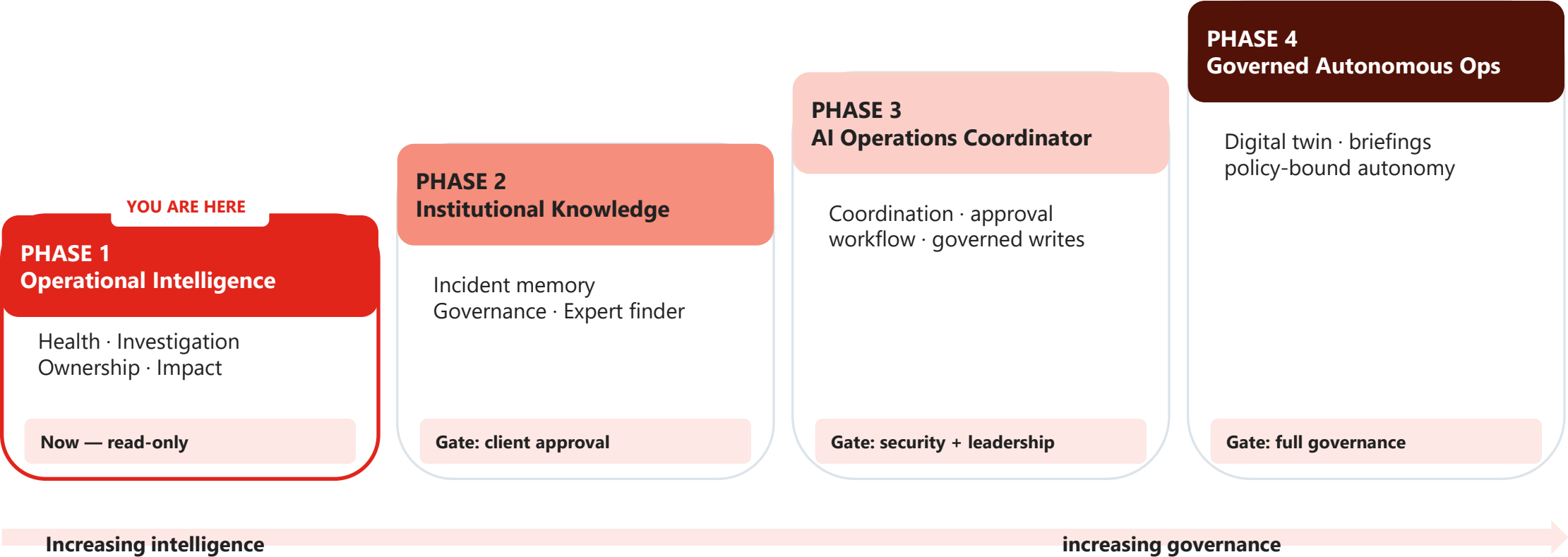
Scope: 1–2 business processes · 5 read-only connectors · 16-table Aurora schema · 4 agents + synthesis

Nothing in EOCC writes back to your systems in Phase 1. It observes, reasons, and advises.

THE TRAJECTORY

Future Vision

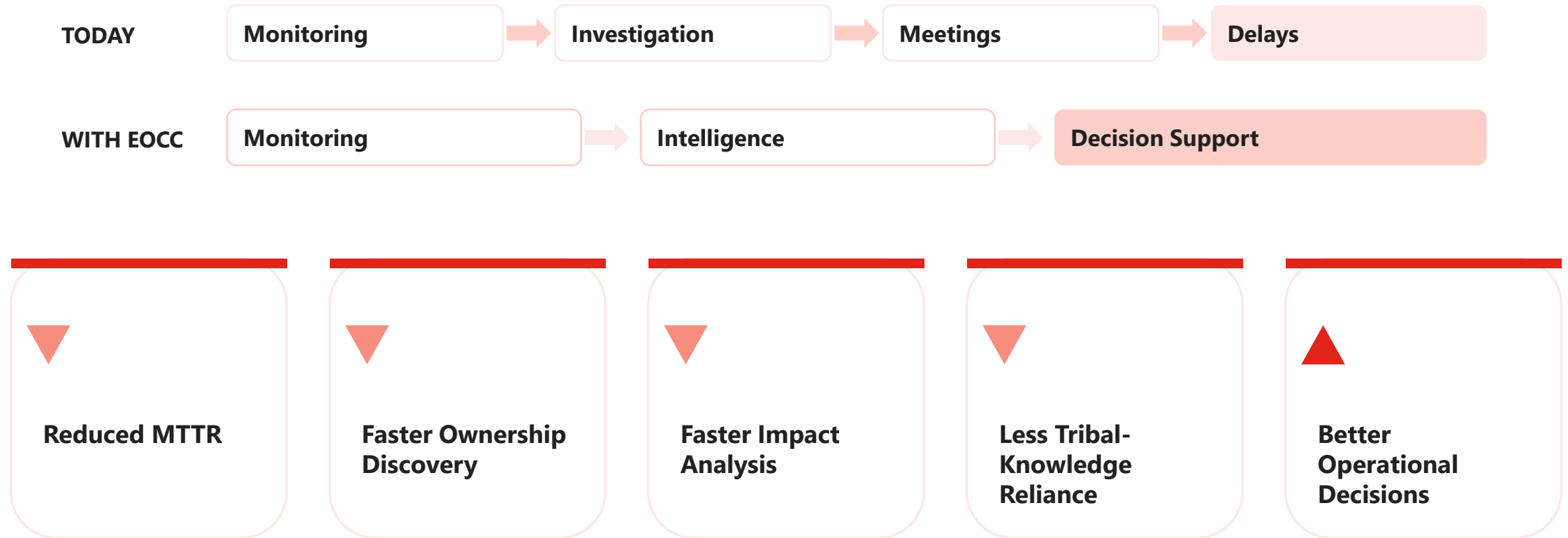
Each phase unlocks the next — gated by approval, never by default.



THE PAYOFF

Why This Matters

The shift: from reacting and meeting, to deciding with evidence.



Same tools. Same data. A decision layer your operations team does not have today.

LET'S VALIDATE

Discussion & Technical Review

Four questions to pressure-test before committing to a pilot.

Q1

Does this solve a real operational challenge?

The 20–40 min ownership hunt; the all-hands bridge call.

Q2

Would this help your operations teams?

Ops engineers, SREs, service owners, team leads.

Q3

Is the architecture viable in your environment?

AWS-native, Cortex AI gateway, read-only connectors.

Q4

What would a pilot require from your side?

1–2 processes, connector credentials, schema seed.

Proposed next step: a scoped, read-only Phase 1 pilot on 1–2 business processes.

Enterprise Ownership Intelligence

Know who owns any service — instantly and with citations — before remediation even begins.

THE PROBLEM

Engineers lose 20–40 minutes during an outage asking “Who owns this Glue job, Lambda, or RDS?” Ownership is tribal knowledge scattered across Slack, Jira, and org charts — escalation paths unclear.

HOW IT WORKS

- 1 Ask in Copilot — “Who owns glue-job-cn3?”
- 2 Router selects the ownership-only path
- 3 Ownership Agent queries the Aurora ownership graph — SQL, not AI guesswork
- 4 Citation-backed ownership card is returned

THE OUTCOME

The owning team, primary POC, manager, and full escalation chain — every fact citation-backed. Time-to-owner drops from minutes to seconds, before a single command is run.

EXAMPLE

A clinical-trial data ingestion Glue job (glue-job-cn3) fails at 2 AM — EOCC instantly returns the Clinical Data Platform team, its POC, and escalation chain, cited.

Business Process Monitoring

Monitor business capabilities, not just servers — health that speaks the language of the business.

THE PROBLEM

Teams monitor servers, not business capabilities. Answering "Is the clinical data pipeline healthy?" means manually checking a dozen services across CloudWatch and Splunk.

HOW IT WORKS

- 1 A 5-minute schedule triggers a rollup
- 2 Worker aggregates CloudWatch + Splunk per business process
- 3 Worst-child-wins sets each process's health
- 4 Command Center shows green / amber / red tiles

THE OUTCOME

Business-capability health with drill-down — "Clinical Data Ingestion: AMBER" down to the failing child service, not infra noise.

EXAMPLE

The Clinical Data Ingestion capability spans 8 AWS services — one Glue or Lambda degradation turns the whole capability amber instead of hiding in a CloudWatch dashboard.

Enterprise Dependency Intelligence

Turn an infra alert into a business-labeled blast radius — before you fail anything over.

THE PROBLEM

Infra alerts don't speak business language. Leadership asks "Does this affect patients or revenue?" while engineers only see "RDS CPU high."

HOW IT WORKS

- 1 A Copilot query or RDS alert triggers
- 2 Router selects the dependency-only path
- 3 Dependency Agent runs a recursive Aurora graph query
- 4 Impact Explorer renders the business-labeled blast radius

THE OUTCOME

Downstream services, applications, and the business process at risk — severity tied to capability, not CPU.

EXAMPLE

Alert on clinical-datalake-rds (the curated clinical data store) → 9 downstream pipelines, 3 dashboards, and the regulatory submission feed flagged at risk — before failover.

AI Incident Investigation

Four agents reason in parallel to hand any engineer a cited, senior-level investigation in one screen.

THE PROBLEM

Root-cause analysis means manually correlating Splunk, CloudWatch, Jira, and Confluence — slow, inconsistent, and dependent on senior engineers.

HOW IT WORKS

- 1 Alert → EventBridge → SQS, or an "Investigate" click
- 2 Four agents gather in parallel (15s)
- 3 Outputs join into a single synthesis step
- 4 Investigation Package is persisted to the workspace

THE OUTCOME

A cited Investigation Package — severity, timeline, owner, the closest past incident, and the exact runbook section.

EXAMPLE

A Glue ETL failure for regulatory reporting → Sev2 package with timeline, owner, similar incident, and runbook §3.2.

Enterprise Knowledge Search

One cited search bar across every system engineers dig through during an incident.

THE PROBLEM

Runbooks live in Confluence, procedures in Jira, tribal knowledge in Slack. Engineers search four systems mid-incident.

HOW IT WORKS

- 1 Search from Copilot
- 2 Resolve the service via external_reference
- 3 Knowledge Agent embeds via Cortex + queries Pinecone
- 4 Ranked docs returned with links and service context

THE OUTCOME

Unified semantic search across Confluence, Jira, and GitHub READMEs — ranked, cited answers in one search.

EXAMPLE

"How do we recover the GxP-validated ingestion pipeline after a Spark out-of-memory error?" → ranked runbook and procedure links, cited.

Historical Incident Intelligence

Stop re-investigating the same failure — surface the past incident and its fix automatically.

THE PROBLEM

Teams repeat investigations for the same failure pattern because past resolutions are buried in closed Jira tickets.

HOW IT WORKS

- 1 Triggered during an active investigation
- 2 Evidence Agent reads the current signals
- 3 Knowledge Agent matches the historical corpus
- 4 Synthesis scores similarity and surfaces the fix

THE OUTCOME

Institutional memory at the point of need — "87% match to INC-4521, resolved by updating the IAM trust policy."

EXAMPLE

The current Glue failure matches INC-4521 from six months ago — same error signature, same IAM role-drift cause.

Autonomous Incident Coordination

From diagnosis to the right people engaged — coordination without manual ticket-wrangling.

THE PROBLEM

After diagnosis, someone still manually creates Jira tickets, finds who's on-call, and notifies stakeholders — slow and inconsistent.

HOW IT WORKS

- 1 Investigation completes with risk $\geq 70\%$
- 2 Coordination Agent picks the engineer (skill + availability)
- 3 Auto-assigns Jira and notifies via Teams
- 4 Optionally opens a war room — human approves

THE OUTCOME

The right people engaged consistently, in seconds. Human approves; the system coordinates. (Requires Teams + Jira write.)

EXAMPLE

Investigation finishes at risk 88% on the overnight clinical-data ingestion pipeline → ticket auto-assigned to the owning data engineer and stakeholders notified, before the morning analytics SLA is missed.

AI Executive Briefings

Translate technical truth into a leadership-ready brief in minutes — not war-room jargon.

THE PROBLEM

Leadership gets either too-technical war-room updates or a vague "we're working on it" — never a business-impact summary.

HOW IT WORKS

- 1 Sev1, or a daily 08:00 schedule
- 2 Load active incident + criticality + business context
- 3 Synthesis runs the executive template
- 4 One-page brief saved as a PDF to S3

THE OUTCOME

A one-page leadership brief stating business impact in business language — generated, not hand-written.

EXAMPLE

A Sev1 on clinical data submission during business hours → the VP gets a one-page brief in 10 minutes.

Architecture Intelligence

A living architecture map with current owners — not a stale Confluence diagram.

THE PROBLEM

Architecture diagrams in Confluence are stale. Troubleshooting means asking the architects who designed the system years ago.

HOW IT WORKS

- 1 Architecture query in Copilot
- 2 Read the Aurora architecture_component tree
- 3 Knowledge Agent RAGs live Confluence + GitHub docs
- 4 Merge into an App → API → DB map with owners

THE OUTCOME

S3 → Glue → Redshift → BI, with current owners attached — the data flow you can actually troubleshoot against.

EXAMPLE

"Show me the clinical data platform — what feeds the warehouse behind the regulatory dashboard?" → a current, owner-tagged data-flow map.

Change Risk Intelligence

A deploy gate that knows the blast radius — proceed or delay, with cited rationale.

THE PROBLEM

Deployments go out without understanding their blast radius on active incidents or dependent business processes.

HOW IT WORKS

- 1 GitHub deploy webhook fires
- 2 Sync the change request
- 3 Dependency blast radius + active-risk cross-check
- 4 Synthesis returns a proceed / delay risk score

THE OUTCOME

A deploy gate with cited rationale — "DELAY: the warehouse is already amber" — preventing a bad situation from getting worse.

EXAMPLE

A deploy webhook for ingestion-glue-job v2.3 fires while the clinical data warehouse is already amber → EOCC recommends DELAY to protect tonight's regulatory load, cited.

Operational Risk Detection

Surface risk before the outage — proactive cards instead of reactive pages.

THE PROBLEM

Teams are reactive — they learn about risk only when something is already down.

HOW IT WORKS

- 1 Governance rules run every 15 minutes
- 2 Evaluate AWS + Splunk signals
- 3 Emit operational_risk_signal records
- 4 Proactive risk cards in the Governance Center

THE OUTCOME

Risk surfaced before it pages — a proactive feed of operational risk, not after-the-fact alerts.

EXAMPLE

Splunk shows a rising dead-letter rate on the clinical-data ingestion queue — flagged before tonight's load fails and breaks the morning regulatory dashboard.

Ownership Gap Detection

Find the orphan resources before they become incident black holes.

THE PROBLEM

Hundreds of AWS resources accumulate without owners after reorganizations — orphans become incident black holes.

HOW IT WORKS

- 1 Nightly 02:00 UTC scan
- 2 Scan the service catalog for missing owner / escalation
- 3 Emit governance_finding records
- 4 Remediation list for the platform team

THE OUTCOME

Closes the #1 ops gap — a clean remediation list so nothing critical is left unowned.

EXAMPLE

A nightly scan finds 34 Glue jobs and Lambdas with no ownership row and no escalation policy — orphan pipelines waiting to fail.

Runbook Coverage Analysis

Recovery readiness as a number — coverage % and the exact gaps.

THE PROBLEM

Tier-1 services lack recovery procedures, so teams improvise during Sev1 incidents.

HOW IT WORKS

- 1 Weekly, after the Confluence sync
- 2 Build the runbook registry
- 3 Match Tier-1/2 services to runbooks
- 4 Flag the no-runbook gaps

THE OUTCOME

Coverage % plus the precise missing-runbook list for SRE leads — recovery readiness, quantified.

EXAMPLE

The nightly clinical-data ingestion pipeline feeds the regulatory submission and is Tier-1, but has no linked recovery runbook.

Documentation Governance

Treat documentation drift as the operational risk it is — flagged, not discovered mid-incident.

THE PROBLEM

Confluence pages for critical systems are years out of date, so engineers follow the wrong procedures.

HOW IT WORKS

- 1 Weekly schedule
- 2 Scan Confluence for stale / missing docs
- 3 Emit stale_doc governance findings
- 4 Report routed to the doc owners

THE OUTCOME

A stale-documentation report for doc owners — GxP-relevant, because drift is operational risk.

EXAMPLE

The batch-submission runbook was last reviewed 18 months ago; the architecture has changed twice since.

Organizational Dependency Analysis

Make cross-team dependencies visible — and end the blame ping-pong.

THE PROBLEM

Technical dependency maps exist, but cross-team dependencies are invisible — so incidents become blame ping-pong.

HOW IT WORKS

- 1 Team view or Copilot query
- 2 Read team_dependency + roll up service deps by team
- 3 Build the cross-team graph
- 4 Render it in the Impact Explorer team view

THE OUTCOME

Upstream and downstream teams made explicit — finger-pointing replaced by a shared map.

EXAMPLE

Data Engineering depends on Security's KMS rotation; when keys rotate, pipelines fail and teams argue ownership.

Critical Expert Detection

Know who the real experts are — and where you're one resignation from trouble.

THE PROBLEM

One engineer solved 25 Aurora incidents; if they leave, the organization loses critical expertise (key-person risk).

HOW IT WORKS

- 1 Copilot query or governance report
- 2 Rank engineer experience from Jira + GitHub
- 3 Surface the top experts per service
- 4 Flag key-person concentration risk

THE OUTCOME

Faster routing today and succession planning tomorrow — expertise made visible, risk made explicit.

EXAMPLE

"Who actually knows Aurora replication for the clinical data store?" → ranked experts, with a key-person flag.

Service Consumption Intelligence

Finally know who consumes your APIs — for chargeback, governance, and safe retirement.

THE PROBLEM

Platform teams don't know who consumes their APIs, so chargeback, governance, and blast-radius planning all suffer.

HOW IT WORKS

- 1 Executive dashboard view
- 2 AWS sync builds the service_consumption map
- 3 Run a reverse-dependency query
- 4 Render consumers + blast radius

THE OUTCOME

A consumer map and reverse dependencies — platform economics and confident retirement planning.

EXAMPLE

"Who consumes the curated clinical dataset — and which dashboards and teams break if we retire the v1 schema?" → a clear consumer map.

Enterprise Operational Search

Google for your operations estate — docs, tickets, and service metadata in one result set.

THE PROBLEM

Search is fragmented —
Confluence search misses Jira; Jira
search misses Splunk-linked
context.

HOW IT WORKS

- 1 Type into the unified search bar
- 2 Parallel semantic search via Pinecone
- 3 Parallel entity search via Aurora
- 4 Merge, dedupe, and group the results

THE OUTCOME

One unified, grouped result set
across the whole estate — no more
four-tab searching.

EXAMPLE

Search "warehouse backfill procedure clinical data" → runbooks, tickets, and service metadata returned together.

Operational Digital Twin

An auditable, point-in-time model of the entire operational estate.

THE PROBLEM

There's no single point-in-time picture of the whole estate for audits, drills, or post-incident review.

HOW IT WORKS

- 1 Hourly schedule
- 2 Full connector sync
- 3 Export the operational graph
- 4 Save a digital_twin_snapshot to S3

THE OUTCOME

A point-in-time operational twin for audits, resilience drills, and post-incident review.

EXAMPLE

A quarterly regulatory resilience review needs every data pipeline, dependency, freshness state, and open risk as-of Friday 5 PM.

Controlled Operational Execution

Speed with governance — recovery actions executed under quorum approval, fully audited.

THE PROBLEM

Recovery actions (restart ECS, scale, rollback) are manual, untracked, and risky without approval.

HOW IT WORKS

- 1 Approve an action from an investigation
- 2 Approval quorum — 2–6 senior approvers for prod
- 3 Control Agent calls the AWS API
- 4 Logged to action_execution_log + Jira comment

THE OUTCOME

Executed actions with a full audit trail — speed without cowboy ops.

EXAMPLE

An investigation recommends re-running the failed nightly Glue ingestion job and scaling the EMR cluster → one-click execute under approval, fully audited, so data lands before the SLA.